EXAONE Tabular 1.0 : Technical Report

A Synthetic-Prior Foundation Model for Tabular Classification and Regression

Moonjung Eo*, Min-Kook Suh*, Hye-Seung Cho*, Jiwon Kim*, Seoyoon Kim*, Sangjun Nam*, Soonyoung Lee
LG AI Research

EXAONE Tabular is a compact tabular foundation model family for classification and regression via in-context learning, producing predictions without dataset-specific gradient updates. Pretrained exclusively on a synthetic structural-causal-model (SCM) prior, its central contribution is an architecture-centered redesign of tabular in-context learning. Rather than compressing features into a fixed row embedding before a separate row-level learner, EXAONE Tabular interleaves feature-axis attention within each item with support-conditioned item-axis attention within each feature at every Transformer layer, mediated by item-summary and feature-summary tokens. Across four public benchmarks, EXAONE Tabular combines strong predictive performance with high efficiency. On TabArena, its 20.81M-parameter classification model ranks first overall, surpassing tuned ensembles and 4-hour AutoML pipelines, while regression reaches the performance regime of the 1.64B-parameter TabFM at roughly 1/11 the inference cost. On BCCO and TALENT, EXAONE Tabular ranks second in classification and first in regression. On ScoringBench, it achieves the best mean rank for both point-estimation and predictive-distribution quality, leading the R^2, RMSE, and CRPS evaluations. Together, these results establish EXAONE Tabular as a state-of-the-art compact tabular foundation model family, combining strong predictive performance across classification, point regression, and probabilistic regression with an efficient model design.

Model: EXAONE Tabular
Date: August 2026
GitHub: https://github.com/LGAI-Research/EXAONE-Tabular
Hugging Face: https://huggingface.co/LG-AI-Research/EXAONE-Tabular

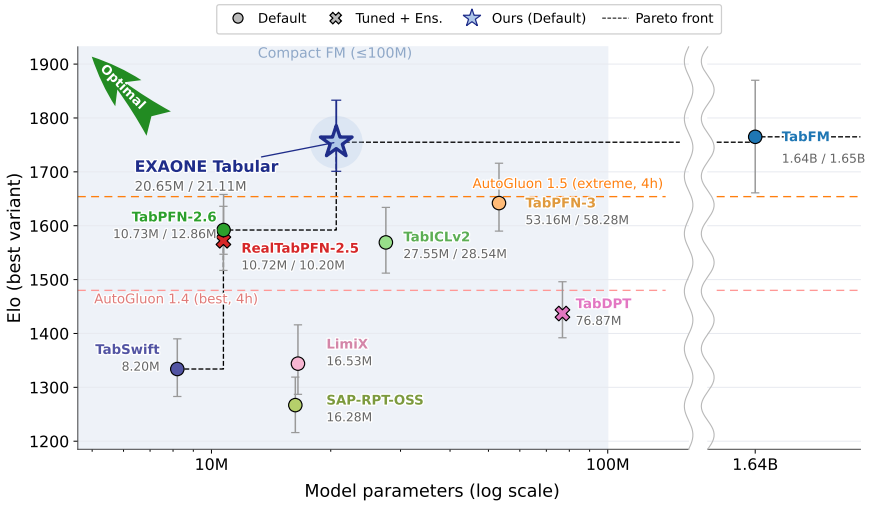


Figure 1: Performance–size Pareto front on the full TabArena benchmark (classification and regression). EXAONE Tabular lies on the Pareto front in its default setting and is the strongest compact tabular foundation model (≤100M parameters), outperforming every model shown except TabFM — with which it is statistically tied at ~1.3% of the parameters — by 113 to 488 Elo. Sizes count the weights stored in the evaluated checkpoints.

*These authors contributed equally.

1 Introduction

Tabular prediction supports operational decisions in clinical risk estimation [1], credit scoring [2], predictive maintenance [3], manufacturing quality control [4], scientific measurement [5], and many other real-world domains [6]. For many years, gradient-boosted decision trees, including XGBoost, LightGBM, and CatBoost, have been the strongest general-purpose default for tabular data [7, 8, 9, 10]. However, applying these models involves separate training for each dataset. Such dataset-specific training requires labeled data and computational resources for every new task, making deployment costly and time-consuming when rapid adaptation across many datasets is required.

Tabular foundation models challenge this convention by pretraining a reusable prediction algorithm across many tasks and applying it to a new dataset through in-context learning [11, 12, 13, 14, 15, 16]. Instead of optimizing a new set of parameters for each dataset, a pretrained model receives a labeled support set together with an unlabeled query set and directly predicts the corresponding query targets. Formally, given a labeled support set

$$D_{\text{support}} = \{(x_i, y_i)\}_{i=1}^{N}, \tag{1}$$

and an unlabeled query set

$$X_{\text{query}} = \{x_j\}_{j=N+1}^{N+M}, \tag{2}$$

the model predicts the query targets $Y_{\text{query}} = \{y_j\}_{j=N+1}^{N+M}$ through

$$p\left(Y_{\text{query}} \mid X_{\text{query}}, D_{\text{support}}\right), \tag{3}$$

without updating its pretrained parameters. By meta-training over a broad family of data-generation mechanisms, prior-fitted models can therefore produce predictions on previously unseen datasets through forward inference rather than dataset-specific gradient-based optimization [11].

EXAONE Tabular is LG AI Research's first tabular foundation model family, comprising separately trained classification and regression models that share the same core architecture and are pretrained from scratch on synthetic data generated from a structural causal model (SCM) prior. At the core of both models is the *Cross-Axis Summary Transformer* (CAST), which maintains a separate representation for each table cell throughout the network. Each CAST layer first applies feature-axis attention within each item and then performs support-conditioned item-axis attention within each feature. Learned item-summary and feature-summary tokens connect these operations: item-summary tokens are concatenated along the feature axis, whereas feature-summary tokens are concatenated along the item axis. By repeating this exchange throughout the network, CAST progressively refines feature interactions using support-set context while retaining cell-level representations. This contrasts with staged architectures that aggregate feature embeddings into a single row representation before dataset-level in-context learning [14, 15, 16].

This report is the first public technical description of EXAONE Tabular, and its main contributions are the following.

- **State-of-the-art compact tabular foundation models.** EXAONE Tabular provides separately trained classification and regression models with approximately 20.8M and 21.1M parameters, respectively, achieving state-of-the-art or near-state-of-the-art performance across four public benchmarks. On TabArena [17], the classification model ranks first among all evaluated methods using a single fixed default configuration, without dataset-specific tuning or post-hoc ensembling, while the regression model reaches the performance regime of the 1.64B-parameter TabFM [18] at roughly 1/11 the inference cost. On BCCO [19] and TALENT [20], EXAONE Tabular ranks second in classification and first in regression. On ScoringBench [21], it achieves the best mean rank for both point-estimation and predictive-distribution quality, leading the R^2, RMSE, and CRPS evaluations.
- **Cross-Axis Summary Transformer (CAST).** We introduce CAST, an interleaved feature–item architecture for tabular in-context learning. Each Transformer layer couples feature-axis processing within items with

support-conditioned item-axis processing within features through distinct item-summary and feature-summary pathways. Unlike staged architectures that compress feature representations before the principal dataset-level ICL stage [14, 15, 16], CAST preserves cell-level representations throughout the hierarchy, allowing feature representations and support-set context to be progressively refined across layers.

- **Strong performance under realistic missing-data conditions.** EXAONE Tabular consumes missing values natively, without requiring imputation, and incorporates missingness information when constructing feature representations. On BCCO [19], which includes a substantial number of datasets with missing entries, the classification model ranks second overall while the regression model ranks first, demonstrating strong predictive performance on realistic incomplete tabular data.

The remainder of this report reviews related work, describes the architecture and synthetic prior in Section 2, presents the training recipe in Section 3, defines the evaluation protocol in Section 4, and discusses limitations and future work in Section 5.

Related Work and Architectural Positioning

EXAONE Tabular follows the synthetic-prior, in-context-learning paradigm established by Prior-Fitted Networks [11] and instantiated for tabular prediction by the TabPFN line of work [12, 13, 22]. TabPFN v1 [12] represents each sample as a single token and performs in-context learning directly over sample embeddings. TabPFN v2 [13] instead maintains cell-level representations and alternates attention along the feature and sample axes, while TabPFN-2.5 [22] extends this architectural line to substantially larger datasets and improved predictive performance.

A second family separates feature representation from dataset-level in-context learning by compressing each row into a fixed-dimensional embedding before a separate ICL stage. TabICL [14] constructs distribution-aware feature representations, models within-row interactions, and aggregates them into one row embedding per sample prior to dataset-level ICL; TabICLv2 [16] extends this design, and TabPFN-3 [15] follows a similar multi-stage organization. TabFM [18], a 1.64B-parameter model pretrained on hundreds of millions of SCM-generated synthetic tables, combines elements of both architectural families. According to its official technical release and model documentation—although no archival architecture paper has been published at the time of writing—TabFM performs repeated cell-level feature- and sample-axis attention, followed by row compression and a separate ICL Transformer. It therefore retains a terminal compression boundary between cell-level contextualization and row-level ICL.

Other recent tabular foundation models differ along the pretraining objective and context-construction axes. TabDPT [23] is pretrained on real tabular data with a masked-column objective and retrieves a bounded, query-relevant context rather than conditioning on the full support set. LimiX [19] retains SCM-based synthetic pretraining but reformulates prediction as masked joint-distribution modeling, expressing classification, regression, imputation, and generation as conditional queries to a single model.

EXAONE Tabular occupies a different position in this design space. Unlike TabICL, TabICLv2, TabPFN-3, and TabFM, it does not impose a one-time compression boundary between feature processing and dataset-level contextualization. Instead, feature-axis processing and item-axis contextualization are interleaved throughout the Transformer hierarchy, allowing support-set context formed at one layer to influence feature representations at subsequent layers. Relative to the TabPFN v2 family, item-axis interaction augments each feature's support sequence with learned feature-summary tokens that persist across Transformer layers. This organization preserves recurrent feature–context interaction while maintaining cell-level states throughout both axis-wise operations.

EXAONE Tabular further maintains item-summary and feature-summary tokens with explicit and complementary roles. Item-summary slots are instantiated for every item and joined to its feature-axis sequence, whereas feature-summary slots are instantiated for every feature and joined to its support item-axis sequence. These pathways preserve cell-level representations throughout the hierarchy rather than converging early into a single fixed `[CLS]`-style row representation processed by a separate ICL module. Together, these design choices constitute the Cross-Axis Summary Transformer (CAST), described in detail in Sec-

tion 2.1. Table 1 summarizes the core components of EXAONE Tabular.

Table 1: Core components of EXAONE Tabular.

Component	Role
Cross-Axis Summary Transformer (CAST)	Interleaves feature-axis processing within items with support-conditioned item-axis processing within features; item-summary tokens join feature-axis sequences, while feature-summary tokens join support item-axis sequences.
Synthetic SCM prior	Generates causal, categorical, nonlinear, noisy, and missing-data prediction tasks without using real pretraining tables.
Task-specific prediction heads	Uses separate classification and regression heads for the independently trained task-specific checkpoints.
Training and inference pipeline	Applies task-specific optimization and inference procedures, including preprocessing, test-time ensembling, and chunked support-conditioned inference.

2 Modeling

EXAONE Tabular is LG AI Research's tabular foundation model family for classification and regression, comprising separately trained task-specific models that share the same core architecture and are pretrained entirely on synthetic data. This section specifies the model architecture (§2.1), prediction heads (§2.2), SCM prior (§2.3), preprocessing and test-time ensembling (§2.4), and inference procedure (§2.5).

2.1 Model Architecture

An overview of EXAONE Tabular's architecture is shown in Figure 2. At the core of EXAONE Tabular is the *Cross-Axis Summary Transformer* (CAST), which jointly processes a table along its feature and item axes while retaining cell-level representations throughout the network. Each Transformer layer applies two feature-axis attention operations within every item, followed by support-conditioned item-axis interaction within every feature, and this pattern is repeated across all 12 layers.

Design Motivation. CAST is designed to keep feature processing and support-set contextualization coupled throughout the Transformer hierarchy. Cross-axis information is routed through two complementary summary states: item-summary tokens (S_i) and feature-summary tokens (S_f). Item-summary tokens are concatenated along the feature axis, whereas feature-summary tokens are concatenated along the item axis. This organization allows feature representations and support-set context to be progressively updated across layers without collapsing the feature axis into a fixed row representation.

Feature Encoding. Each input feature is encoded independently. Finite input values and indicators for missing or infinite values are jointly processed by a dedicated missing-value handling step. The model can therefore use both observed values and missingness patterns when constructing feature representations.

Feature-Axis Processing with Item-Summary Tokens. Before feature-axis self-attention, each cell reads the feature-summary states attached to its feature through cross-attention and incorporates the result through a residual connection. For every item, three item-summary tokens are then concatenated with its cell states along the feature axis. Self-attention over this combined sequence jointly updates the item-summary tokens and cell states. EXAONE Tabular repeats this feature-axis operation twice per layer without collapsing the feature axis into a fixed item embedding, so cell-level representations remain available throughout the Transformer hierarchy.

Item-Axis Processing with Feature-Summary Tokens. Before item-axis attention, each cell reads the three item-summary states attached to its item through cross-attention and incorporates the result through a

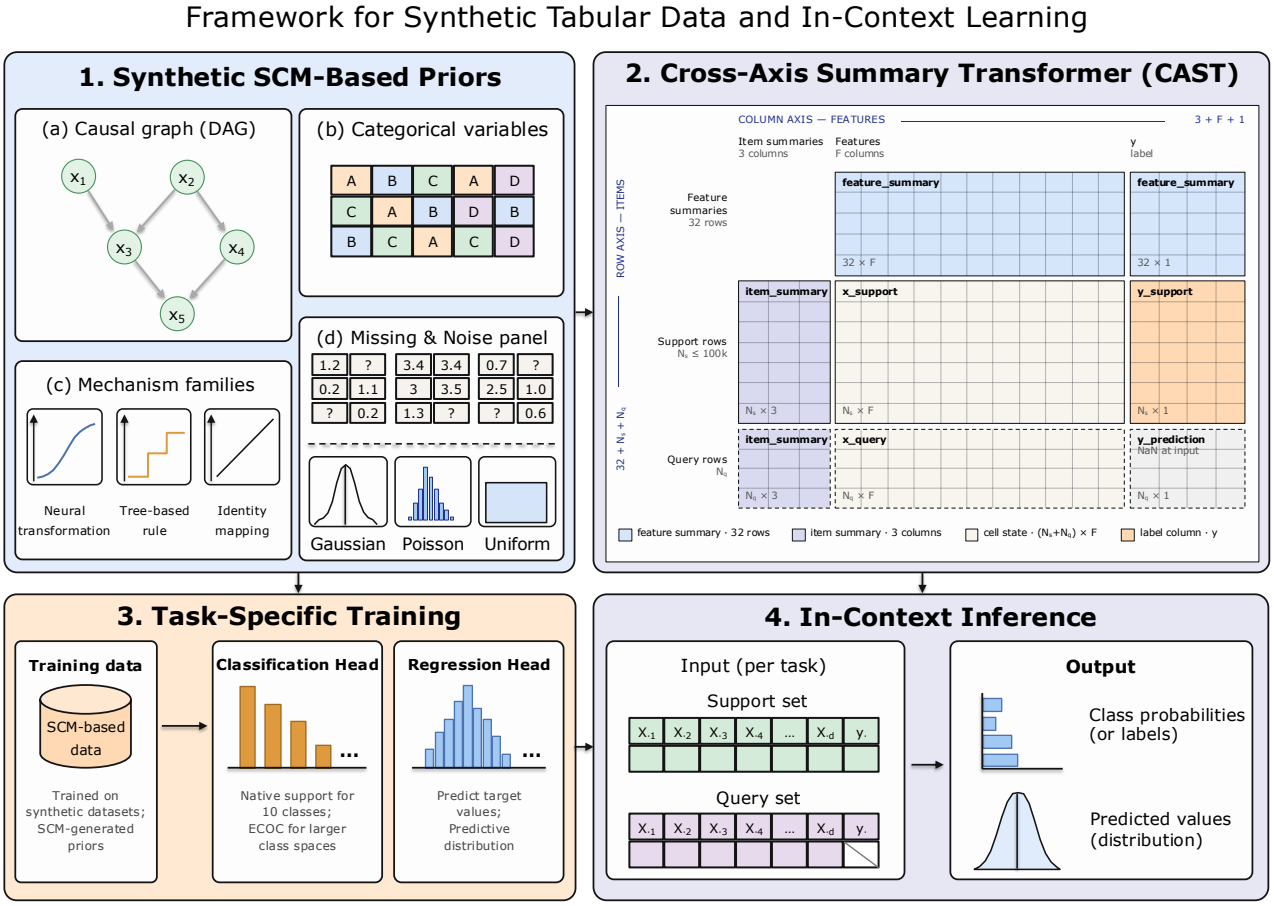


Figure 2: Overall framework of EXAONE Tabular. The model combines an SCM-based synthetic prior, a `Cross-Axis Summary Transformer` with interleaved feature-axis and item-axis attention, task-specific heads for classification and regression, and in-context inference on support and query sets. Item-summary tokens are concatenated with cell states along the feature axis, while feature-summary tokens are concatenated with support-item states along the item axis.

residual connection. For every feature, 32 feature-summary tokens are then concatenated with the support-item cell states along the item axis. The attention computation is divided into two paths:

- **Support path:** the feature-summary tokens and support-item cell states are jointly updated through self-attention over their combined item-axis sequence.
- **Query path:** each query-item cell cross-attends to the same combined feature-summary and support-item input sequence.

Query items neither attend to one another nor update the feature-summary tokens or support-item states. The item-axis context available to every query is therefore determined exclusively by the labeled support set and the feature-summary states entering the layer. Consequently, the prediction for an individual query row can be written as

$$p(y_j \mid x_j, D_{\text{support}}),$$

without dependence on other query examples in the same inference batch. This conditional independence enables query examples to be processed in separate chunks without changing the support-conditioned prediction, apart from minor numerical differences introduced by the underlying computation kernels.

In short, item-summary tokens are concatenated with each item's cells along the feature axis, whereas feature-summary tokens are concatenated with each feature's support-item cells along the item axis.

Scalable Attention Normalization. EXAONE Tabular uses Scalable Softmax (SSMax), which adjusts at-

tention normalization according to the number of context tokens. Standard softmax attention can become increasingly diffuse as context length grows; SSMax was introduced to counteract this effect and improve context-length generalization [24].

EXAONE Tabular uses a 192-dimensional embedding, 6 attention heads, and 12 Transformer layers. The classification checkpoint contains 20,807,866 parameters (approximately 20.81M), comprising approximately 20.65M backbone parameters and 0.156M task-head parameters. The regression checkpoint uses the same backbone together with a distributional regression head, resulting in approximately 21.11M parameters in total. Table 2 summarizes the core architectural configuration.

Table 2: Core model configuration.

Configuration	**Value**
Embedding dimension	192
Attention heads	6
Transformer layers	12
Feed-forward expansion	4×
Feature-attention operations per layer	2
Item-summary tokens per item	3
Feature-summary tokens per feature	32
Attention normalization	SSMax
Total parameters, classification	20.81M
Total parameters, regression	21.11M

2.2 Prediction Heads

Classification. The native classification head is a two-layer MLP decoder that maps the representation of each query row to logits over up to 10 classes, with class probabilities obtained through softmax. Datasets with more than 10 classes are handled at inference time through an Error-Correcting Output Code decomposition, described in Section 2.5.

During test-time ensembling, class-label indices may be permuted to reduce sensitivity to the arbitrary numerical ordering of class labels.

Regression. The regression head directly predicts 999 conditional quantiles for each query item. Specifically, it outputs estimates

$$\hat{Q}(\tau_k \mid x_j, D_{\text{support}}), \qquad \tau_k = \frac{k}{1000}, \quad k = 1, \ldots, 999,$$

For point prediction, the default inference procedure first sorts the predicted quantiles to guard against quantile crossing and then computes a trapezoidal average over the central 99.8% of the quantile function. This provides an approximation to the conditional mean. The median, corresponding to the $\tau = 0.5$ quantile, is also available as an alternative point-estimation readout.

2.3 Synthetic SCM Prior

EXAONE Tabular is pretrained without real tabular datasets. Each training episode is generated from a synthetic structural causal model (SCM) prior designed to cover a broad range of tabular data-generating processes. The generator is organized into five stages as follows.

(i) Sample task hyperparameters. The generator first samples high-level task settings, including the number of rows, number of features, number of classes, graph size, component structure, and mechanism families. During pretraining, task size and structure are sampled over a broad range of support-set sizes, feature counts, class counts, and graph configurations, subject to a per-task compute budget.

(ii) Sample a DAG and node noise. Conditioned on the sampled task settings, the generator constructs a directed acyclic graph that defines dependencies among latent variables. The graph may contain one or two weakly connected components, and a randomized topology procedure diversifies the dependency structure across tasks. Each edge is assigned one of three mechanism families: a neural transformation, a tree-based rule, or an identity mapping. Independent noise is sampled per node and injected during SCM computation.

(iii) Compute the SCM in topological order. A topological ordering of the DAG determines the evaluation sequence. Root nodes are initialized from exogenous random draws, after which each remaining node is computed from its parents by a scaled sum of per-edge mechanism outputs. Neural mechanisms apply smooth nonlinear transformations, while tree-based mechanisms apply piecewise, threshold-like rules, with optional per-node Gaussian noise. This process produces fully evaluated SCMs containing diverse nonlinearities, interactions, and noise levels.

(iv) Choose observed features and target. After all SCM variables have been computed, a subset of nodes is selected as observed input features and another node is selected as the prediction target. The resulting rows are assembled into a tabular dataset and divided into labeled support examples and query examples. Classification and regression episodes use the same general generation pipeline while differing in target construction and task-specific sampling settings.

(v) Post-process and filter. The extracted table is transformed to reproduce common characteristics of real tabular data. Post-processing includes categorical discretization, nonlinear feature warping, feature-wise quantization, and task-specific missingness injection. An optional quality filter can screen out tasks that are too easy, too noisy, or insufficiently informative. The resulting dataset forms a synthetic training episode for in-context pretraining.

2.4 Preprocessing and Test-Time Ensembling

The inference pipeline provides a scikit-learn-compatible interface through `fit` and `predict`. In this interface, `fit` registers and preprocesses the labeled support set; it does not update the pretrained model parameters.

When the number of support rows exceeds the configured inference limit, the support set is subsampled before preprocessing. At inference time, predictions can be ensembled across estimators constructed with different preprocessing and permutation configurations. Candidate feature transformations include raw features, robust scaling, power transformations, and quantile transformations. Categorical encodings, feature orderings, and class-label indices may also be permuted.

Both classification and regression support test-time ensembling. For regression, predictions from individual estimators are aggregated across preprocessing and permutation configurations. The default number of test-time estimators is eight for classification, and the ensemble size can be adjusted according to latency and accuracy requirements.

2.5 Inference

The complete inference pipeline may combine multiple preprocessing and permutation estimators, query chunks, and, for classification tasks beyond the native class limit, multiple ECOC subproblems. A complete prediction request may therefore involve multiple model forward operations, although no dataset-specific parameter updates are performed.

Error-Correcting Output Code Decomposition. For classification datasets with more than 10 classes, the inference pipeline uses an Error-Correcting Output Code (ECOC) decomposition [25] to extend the fixed-size classification head to larger label spaces. Each row of the code matrix relabels the original classes into

an alphabet of up to 10 symbols, defining multiclass classification subproblems. All subproblems are evaluated using the same pretrained model, and their output probabilities are decoded into probabilities over the original classes using the code matrix.

Inference Efficiency and Chunking. Inference can be organized along the estimator, ECOC-subproblem, and query dimensions. Estimators are independent because they may apply different transformations or permutations to the support set, while ECOC subproblems use different support-label assignments. Consequently, support-conditioned states generally cannot be shared across estimators or ECOC subproblems.

For a fixed estimator and ECOC subproblem, the support set remains unchanged across query chunks. The model can therefore reuse support-side keys and values when such reuse is compatible with the feature encoding applied to the chunks. The inference executor automatically selects between cached and uncached execution according to the input configuration, device, and available memory. With cached execution, the support-side keys and values are constructed once and reused across query chunks. With uncached execution, each query chunk is evaluated together with the support set and the corresponding support-side values are recomputed.

The attention implementation dispatches between FlashAttention [26] and Memory-Efficient Attention [27] according to the sequence length and tensor configuration. The default compute dtype is fp16, while query-axis and estimator-axis chunking are used to control peak memory consumption for larger support and query sets.

Cost Structure and Support Recomputation. For a single estimator and ECOC subproblem, the computation is dominated by three operations. Feature-axis attention is linear in the number of items and quadratic in the number of feature groups. Item-axis support self-attention scales approximately quadratically with the support size N_s, while query-to-support cross-attention scales with the product of the query size N_q and the support size. The leading item-axis cost is therefore approximately

$$\mathcal{O}\left(N_s^2 + N_s N_q\right)$$

when the support context is constructed once.

The number of distinct support-conditioned states required for one prediction request is

$$E \cdot R,$$

where E is the number of ensemble estimators and R is the number of ECOC subproblems, with $R = 1$ when ECOC decomposition is not required. These factors are inherent because each estimator may use a differently transformed support set and each ECOC subproblem changes the support-label assignment.

If the query set is divided into n_{qc} chunks, the number of estimator–subproblem–query-chunk evaluations is

$$E \cdot R \cdot n_{qc}.$$

When cached execution is selected, the support-side keys and values are constructed

$$E \cdot R$$

times and reused across query chunks. Under uncached execution, they are instead constructed

$$E \cdot R \cdot n_{qc}$$

times. Thus, query chunking preserves the total query-to-support attention cost up to chunking overhead, but uncached execution additionally repeats the support-side construction for each chunk. The value of n_{qc} and the use of caching are determined at inference time rather than fixed by the default configuration.

3 Training

Classification and regression share the main architecture but use separately configured training recipes. Training is performed entirely on synthetically generated tables using bf16 precision. Across the training lineage used for the released models, classification processes approximately 30 million synthetic table instances, while regression processes approximately 10 million. Here, the number of tables refers to the total number of synthetic table instances processed during optimization.

3.1 Optimization

Selected matrix-valued parameters, including most attention and feed-forward weight matrices, are optimized with Muon [28], which applies a momentum-based update followed by matrix orthogonalization. The remaining parameters are optimized with AdamW [29]. The optimizer assignments are therefore

$$\theta_{\text{matrix}} \leftarrow \text{Muon}, \qquad \theta_{\text{remaining}} \leftarrow \text{AdamW}. \tag{4}$$

Classification learning rates are 1×10^{-4} for AdamW parameters and 2×10^{-4} for Muon parameters. For regression, the corresponding learning rates are 3.8×10^{-5} and 2×10^{-4}, respectively. Learning rates and weight decay are adjusted in subsequent training stages.

3.2 Learning-Rate Schedule

Training primarily uses Warmup-Stable-Decay (WSD) learning-rate schedules [30]. WSD consists of an initial warmup phase, an extended stable phase, and a terminal decay phase. Additional continuation and adaptation training uses closely related cosine or constant-learning-rate schedules depending on the training objective.

Exponential Moving Average Both classification and regression maintain EMA model weights throughout training. For parameters θ_t at optimization step t, the EMA parameters are updated as

$$\theta_t^{\text{EMA}} = \gamma \theta_{t-1}^{\text{EMA}} + (1-\gamma)\theta_t. \tag{5}$$

A decay of $\gamma = 0.999$ is used for most training, with stage-specific adjustments in later classification training. Evaluation and inference use the EMA weights.

4 Evaluation

4.1 Benchmarks

We evaluate EXAONE Tabular on four public benchmarks: TabArena [17], BCCO (Balanced Comprehensive Challenging Omni-domain) [19], TALENT [20], and ScoringBench [21]. The four benchmarks cover complementary evaluation regimes and are reported separately rather than pooled into a single heterogeneous collection. TabArena enables a broad comparison with recent tabular foundation models and established task-specific methods. BCCO complements this evaluation by including datasets with missing values, thereby reflecting more realistic tabular data conditions. TALENT further broadens the evaluation with a substantially larger collection of classification and regression datasets spanning diverse tabular tasks. ScoringBench adds a regression-only evaluation that assesses both conventional point estimates and full predictive distributions through proper scoring rules.

For classification, TabArena contains 30 binary and 8 multiclass datasets, while BCCO contains 71 binary and 35 multiclass datasets. TALENT contains 200 classification datasets, including 120 binary and 80 multiclass datasets. To ensure a common comparison set across models with different class-count limits, we exclude 12 TALENT datasets with more than 10 target classes. BCCO additionally contains 50 regression datasets, while TALENT contains 100 regression datasets. ScoringBench is continuously maintained: its original release reports 97 regression datasets, while the live configuration accessed in August 2026 enumerates 104 candidate

datasets. Because dataset validation, model-result coverage filtering, and complete-case selection are applied separately for each metric, the effective number of datasets used in a leaderboard comparison may be smaller and can differ across metrics.

On TabArena, EXAONE Tabular is compared with 13 baselines spanning recent tabular foundation models, gradient-boosted decision trees, neural tabular architectures, and AutoML systems. For BCCO and TALENT, we conduct a controlled comparison against six tabular foundation models: TabSwift [31], TabDPT [23], TabPFN-3 [15], TabICLv2 [16], LimiX [19], and TabFM [18]. For ScoringBench, we compare against all methods eligible for each metric on the official live leaderboard at the time of access. Its comparison set is therefore metric-dependent, particularly for distributional metrics that require a full predictive distribution.

4.2 Evaluation Protocol

TabArena Protocol For TabArena, we use the results reported on the official leaderboard rather than rerunning each comparison model. EXAONE Tabular is evaluated using the same datasets, splits, metrics, and aggregation protocol to ensure direct comparability with the leaderboard results. The comparison covers the full set of leaderboard methods, spanning pretrained tabular foundation models, tree-based methods, neural networks, and AutoML systems; baseline-specific preprocessing, hyperparameter optimization, and ensembling therefore follow the TabArena evaluation protocol. Classification and regression performance are measured with the per-task TabArena metrics and aggregated into Elo ratings following the same protocol. All TabArena Elo values, confidence intervals, and timing measurements reported here (Figures 1 and 3–5) are taken from the official leaderboard (accessed August 2026).

BCCO and TALENT Protocol For BCCO and TALENT, all comparison models were evaluated using their official GitHub repositories and publicly released checkpoints. We preserved each model's official preprocessing and inference procedures whenever possible. To control computational cost across datasets of different scales, the training or in-context support set was capped at 50,000 samples per fold for both classification and regression. When the training split exceeded this limit, 50,000 samples were drawn without replacement, while the validation and test splits remained unchanged.

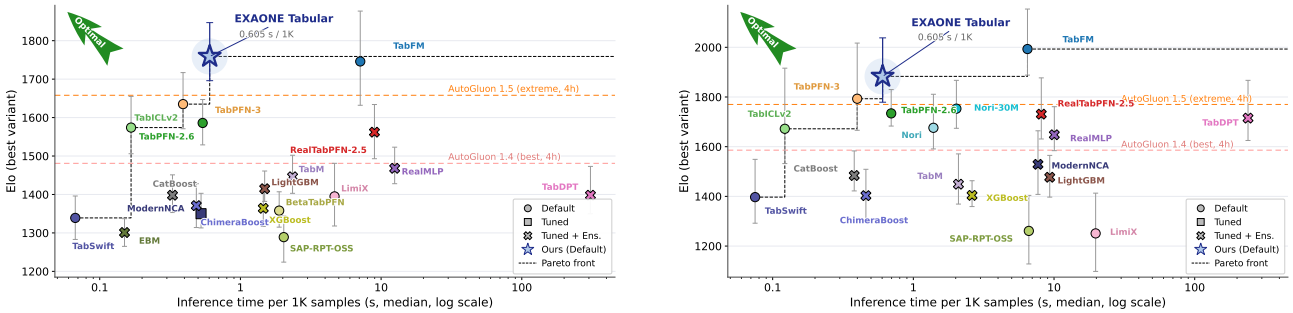
All data-dependent preprocessing was fitted exclusively on the training split and then applied unchanged to the validation and test splits. For models with native categorical-feature support, we retained the default preprocessing pipeline. Models requiring numerical inputs used ordinal mappings fitted on the training data, with categories unseen at test time treated as missing values. When the number of features exceeded a model's input limit, the input dimensionality was reduced according to its official implementation.

For classification, the same set of datasets was used across all comparison models. In TALENT, 12 datasets with more than 10 target classes were excluded because some of the evaluated tabular foundation models do not support larger numbers of classes.

ScoringBench Protocol For ScoringBench, we use the results reported on the official live leaderboard. The benchmark evaluates probabilistic regression using five-fold cross-validation with a fixed random seed and at most 3,000 observations per dataset. Scores are first averaged across folds for each model and dataset, and models are then compared by their mean dataset-level ranks. Point-estimation quality is measured using R^2 and RMSE, while CRPS evaluates the complete predictive distribution rather than only its mean. The leaderboard applies metric-specific coverage filtering, so the eligible models and effective dataset set may differ across metrics. All ScoringBench rankings reported in this report are taken from the official leaderboard (accessed August 2026).

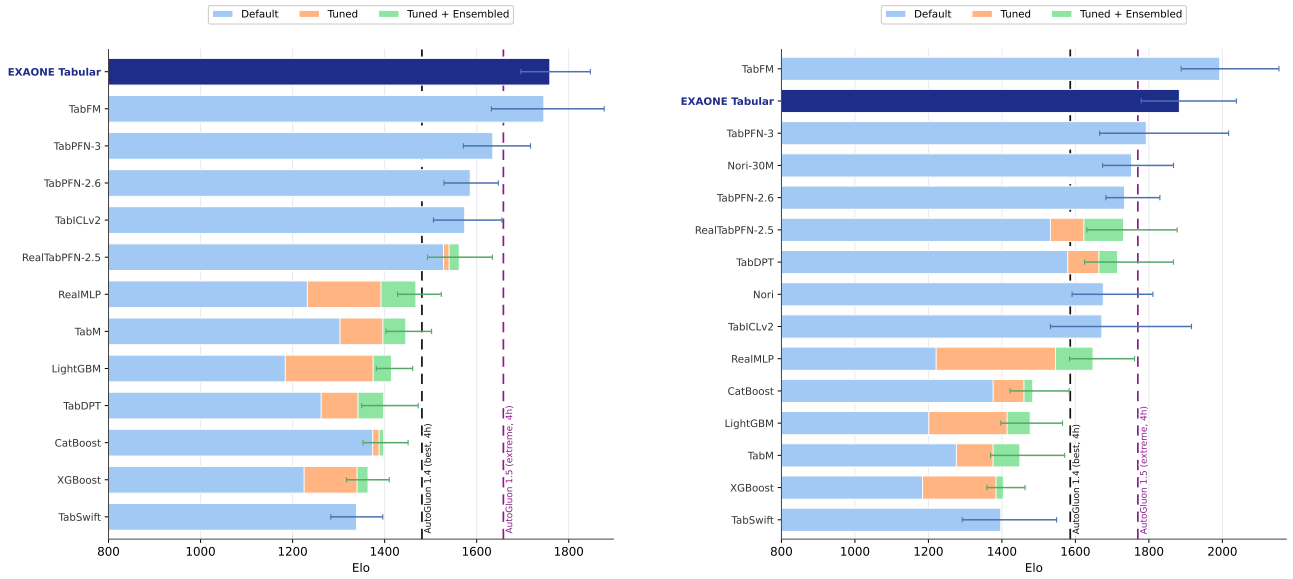
4.3 TabArena Results

TabArena is a continuously maintained benchmark for tabular machine learning that provides curated datasets, standardized evaluation pipelines, and strong implementations of classical, neural, and foundation-model baselines [17]. TabArena comprises 51 real-world predictive tasks—38 classification and 13 regression tasks—



(a) Classification task **(b)** Regression task

Figure 3: Accuracy–latency Pareto fronts on TabArena for (a) classification and (b) regression tasks. Elo is plotted against median inference time per 1,000 samples (log scale); dashed staircases trace the Pareto fronts, and horizontal lines mark the 4-hour AutoGluon reference pipelines [32]. Using its default single-forward-pass inference at 0.605 s per 1,000 samples, EXAONE Tabular lies on the Pareto front of both task types. On classification it attains the highest Elo on the entire leaderboard, exceeding TabPFN-3 by ~125 Elo at comparable latency. On regression it reaches the accuracy regime of TabFM — within overlapping 95% confidence intervals — at roughly 1/11 the inference cost, and exceeds the regression-specialized Nori models [33] by ~140 Elo.



(a) Classification task **(b)** Regression task

Figure 4: Default vs. tuned and post-hoc ensembled TabArena Elo for (a) classification and (b) regression. Bars show each method's default configuration, with additional segments marking the gains from per-task tuning and post-hoc ensembling where those variants exist. Conventional GBDT and neural baselines depend heavily on both, worth ~230 Elo for LightGBM [8] and RealMLP [34] on classification and over 420 Elo for RealMLP on regression alone. EXAONE Tabular (navy), like most tabular foundation models, is evaluated in a single untuned default configuration, and ranks first on classification and, on regression, second only to TabFM — a model ~78× its size. Dashed lines mark the 4-hour AutoGluon 1.4/1.5 pipelines, both of which it clears on both task types.

manually selected from 1,053 candidate datasets. The benchmark covers IID datasets with 500 to 250,000 training samples and substantial variation in feature dimensionality and categorical-feature prevalence, providing a diverse test bed for tabular learning methods.

For the accuracy–latency comparison, each method is represented by its best-performing non-imputed variant on the leaderboard, with marker styles distinguishing default, tuned, and tuned-and-ensembled configurations; inference times correspond to the plotted variant. EXAONE Tabular is evaluated with its single default configuration, refit on the full outer-training set, while baseline inference times are taken from the measurements released by TabArena, so its untuned default is compared against the strongest available configuration of every baseline. Figure 3 places these configurations on the accuracy–latency plane, separately

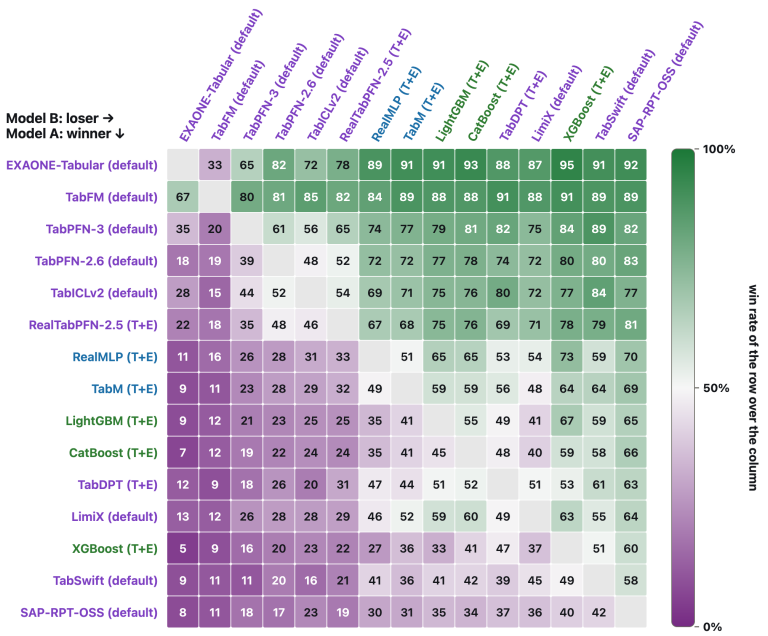


Figure 5: Pairwise win rates on the full TabArena benchmark (classification and regression). Each entry (A, B) reports the percentage of evaluation splits on which method A attains a lower test error than method B; cells are shaded from purple (row loses) through white (parity) to green (row wins). Method labels are colored by family — purple: tabular foundation models; blue: neural networks; green: GBDTs. With a single fixed default configuration, EXAONE Tabular wins the majority of splits against the strongest variant of every competing method except TabFM: 65–92% against the other tabular foundation models, and at least 89% against every tuned-and-ensembled GBDT and neural-network baseline. The advantage is therefore consistent at the split level rather than being driven by a small subset of datasets.

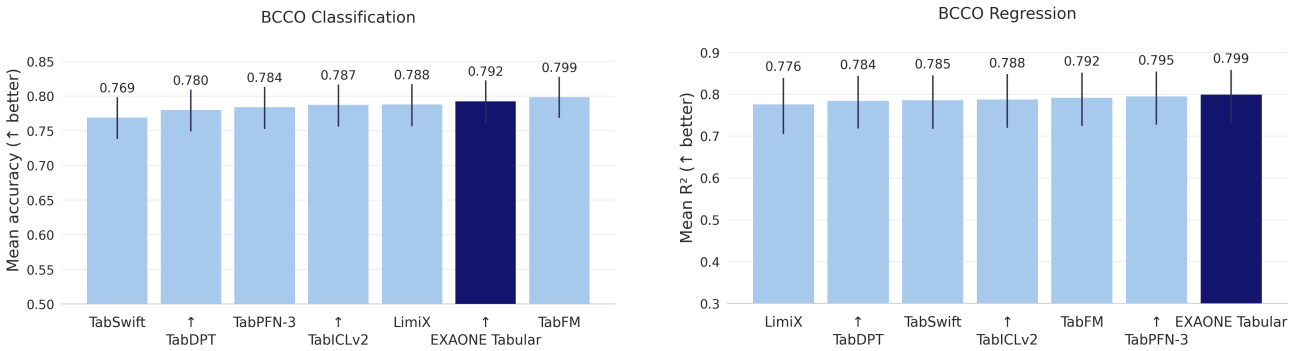
for classification and regression. Using its default single-forward-pass inference at 0.605 s per 1,000 samples, EXAONE Tabular lies on the Pareto front of both task types: on classification it attains the highest Elo on the entire leaderboard, outperforming TabPFN-3 by ∼125 Elo at comparable latency, and on regression it reaches the accuracy regime of TabFM — the strongest evaluated model — at roughly 1/11 the inference cost. TabFM requires approximately 7.0 s per 1,000 samples, an ∼11.5× increase over EXAONE Tabular.

Figure 4 contrasts the value of inference-time tuning and post-hoc ensembling across methods. Conventional GBDT and neural-network baselines climb the leaderboard only through hours of per-task tuning and post-hoc ensembling, whereas most tabular foundation models, including EXAONE Tabular, are evaluated in a single untuned default configuration. Even within this default-only setting, EXAONE Tabular reaches the top tier of both task types — and the converse also holds: however large the tuning and ensembling gains, they never close the gap, as every tuned-and-ensembled variant on the leaderboard still falls short of its single untuned configuration on both tasks.

The pairwise analysis in Figure 5 complements the aggregate Elo ranking by measuring head-to-head consistency across evaluation splits. Elo limits the influence of unusually large performance margins by reducing each comparison to a win, tie, or loss, but a single aggregate rating can still mask intransitive reversals in which a lower-ranked method prevails in direct comparison. The matrix provides no evidence of such reversals for EXAONE Tabular: no method ranked below it wins the majority of splits against it, and its win rates rise broadly with the Elo gap to each opponent. The consistently high win rates also indicate that its advantage is observed across the benchmark rather than being attributable to a small subset of evaluation splits.

4.4 BCCO Results

Classification Figure 6a reports the mean test accuracy of the seven evaluated models across the 106 BCCO classification datasets. TabFM achieves the highest mean accuracy of 0.799, followed closely by EXAONE Tabular with 0.792. EXAONE Tabular outperforms LimiX, TabICLv2, TabPFN-3, TabDPT, and TabSwift, which



(a) Classification performance on the BCCO benchmark, measured by mean test accuracy across 106 datasets.

(b) Regression performance on the BCCO benchmark, measured by mean R^2 across 50 datasets.

Figure 6: Performance comparison on the BCCO benchmark. EXAONE Tabular achieves the second-highest mean accuracy on classification and the highest mean R^2 on regression, demonstrating competitive performance across both tasks. Error bars represent 95% bootstrap confidence intervals.

achieve mean accuracies of 0.788, 0.787, 0.784, 0.780, and 0.769, respectively, while the absolute accuracy gap to TabFM is only 0.007.

This comparison highlights the favorable performance-to-parameter trade-off of EXAONE Tabular. With 20.81M parameters, it is smaller than TabPFN-3 and TabDPT, comparable in scale to TabICLv2, and substantially smaller than the 1.64B-parameter TabFM. Notably, TabFM is evaluated with 32 ensemble members, whereas EXAONE Tabular and the other comparison models use 8. Despite the larger model size and ensemble budget of TabFM, EXAONE Tabular remains within 0.007 in mean accuracy. Although LimiX and TabSwift use fewer parameters, they achieve lower mean accuracy. These results indicate that EXAONE Tabular provides a favorable balance between predictive performance, model size, and inference complexity.

The results also show that EXAONE Tabular performs consistently across the diverse BCCO datasets. Since BCCO includes a substantial number of datasets containing NaN values, its second-best mean accuracy suggests that the model remains competitive under realistic incomplete-data conditions.

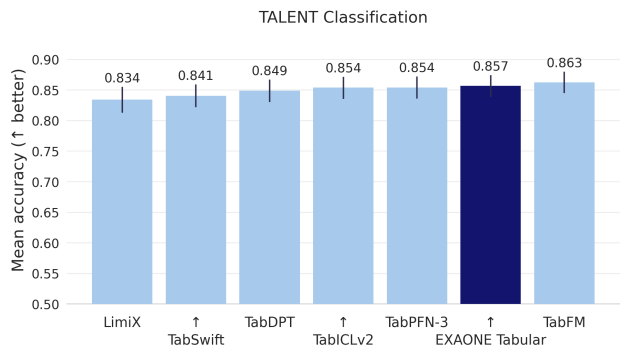
Error bars represent 95% bootstrap confidence intervals obtained by resampling datasets 10,000 times. As the confidence intervals of TabFM and EXAONE Tabular overlap, the observed difference should not be interpreted as statistically significant without an additional hypothesis test. Nevertheless, EXAONE Tabular achieves performance close to TabFM while using only approximately 1.3% of its parameters and one-quarter of its ensemble members, demonstrating a strong balance between predictive performance and inference efficiency.

Regression Figure 6b reports the mean R^2 of the seven evaluated models across the 50 BCCO regression datasets. EXAONE Tabular achieves the highest mean R^2 of 0.799, followed by TabPFN-3 with 0.795. TabFM and TabICLv2 achieve 0.792 and 0.788, respectively, while TabSwift, TabDPT, and LimiX obtain 0.785, 0.784, and 0.776.

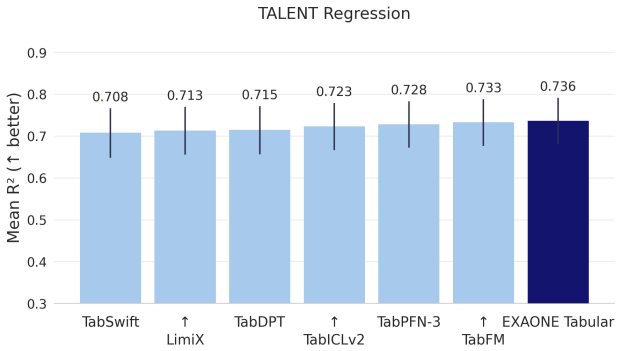
These results show that EXAONE Tabular remains competitive across both classification and regression tasks on BCCO. In particular, EXAONE Tabular ranks first in mean R^2 on regression, complementing its strong classification performance and demonstrating consistent predictive capability across diverse real-world tabular tasks.

4.5 TALENT Results

Classification Figure 7a reports the mean test accuracy of the seven evaluated models across 188 TALENT classification datasets. We exclude 12 of the 200 classification datasets with more than 10 classes to ensure a common comparison set across models with different class-count limits. TabFM achieves the highest mean accuracy of 0.863, followed by EXAONE Tabular with 0.857. TabPFN-3 and TabICLv2 both achieve 0.854,



(a) Classification performance on the TALENT benchmark, measured by mean test accuracy across 188 datasets.



(b) Regression performance on the TALENT benchmark, measured by mean R^2 across 100 datasets.

Figure 7: Performance comparison on the TALENT benchmark. EXAONE Tabular achieves the second-highest mean accuracy on classification and the highest mean R^2 on regression, demonstrating consistently competitive performance across both tasks. Error bars represent 95% bootstrap confidence intervals.

while TabDPT, TabSwift, and LimiX obtain 0.849, 0.841, and 0.834, respectively.

Although TabFM achieves the best overall result, the absolute accuracy gap to EXAONE Tabular is only 0.006. EXAONE Tabular outperforms all remaining tabular foundation models, demonstrating strong and consistent classification performance across the large and diverse TALENT benchmark.

Together with the results on TabArena and BCCO, the TALENT evaluation further shows that EXAONE Tabular remains competitive across independently constructed public benchmarks with different dataset compositions and evaluation regimes. Its strong performance on TALENT provides additional evidence of robust cross-benchmark generalization.

Regression Figure 7b reports the mean R^2 of the seven evaluated models across the 100 TALENT regression datasets. EXAONE Tabular achieves the highest mean R^2 of 0.736, followed by TabFM and TabPFN-3 with 0.733 and 0.728, respectively. TabICLv2, TabDPT, LimiX, and TabSwift obtain 0.723, 0.715, 0.713, and 0.708, respectively.

These results show that EXAONE Tabular maintains strong regression performance across the large and diverse TALENT benchmark. Together with its second-best classification performance, the leading mean R^2 demonstrates that EXAONE Tabular remains consistently competitive across both classification and regression tasks. Combined with the results on TabArena and BCCO, the TALENT evaluation further supports the model's ability to generalize across independently constructed public benchmark suites.

4.6 ScoringBench Results

ScoringBench complements TabArena, BCCO, and TALENT by evaluating an aspect of regression performance that is not captured by point estimates alone. Whereas R^2 and RMSE assess the quality of a scalar prediction derived from the predictive distribution, CRPS evaluates the complete predictive cumulative distribution. The benchmark therefore tests whether a model that provides accurate point predictions also represents the uncertainty and dispersion of possible target values appropriately.

Point Estimation Figure 8 jointly compares the official leaderboard mean ranks for R^2 and CRPS. Along the R^2 axis, EXAONE Tabular achieves the best mean rank, indicating the strongest aggregate point-estimation performance across the evaluated regression datasets. It also ranks first under RMSE, showing that the result is consistent across the two squared-error-based point-prediction metrics rather than being specific to the normalization used by R^2.

Predictive Distribution The same figure shows that EXAONE Tabular also achieves the best mean rank under CRPS. Unlike R^2 and RMSE, which evaluate the predictive mean, CRPS assesses the full predictive

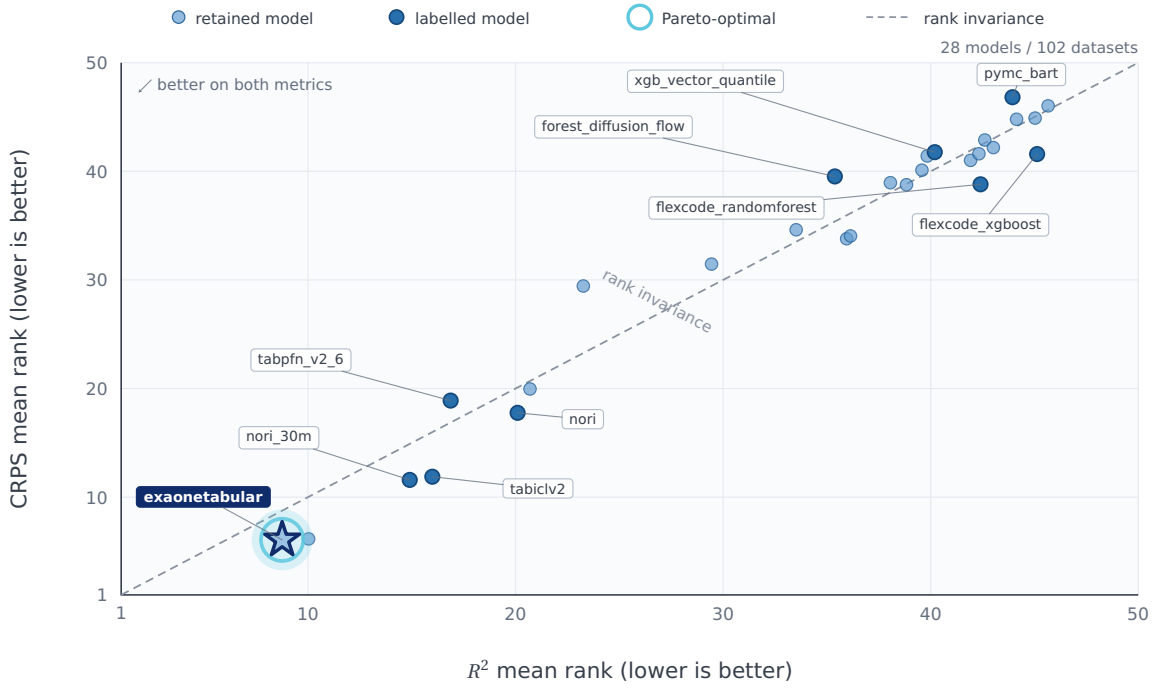


Figure 8: Joint point-estimation and predictive-distribution rankings on ScoringBench. Each point represents a model with valid results for both metrics and is positioned by its official Autorank mean ranks over 102 regression datasets; lower ranks are better on both axes. Fine-tuned and HPO variants are omitted for readability, leaving 28 models. The dashed diagonal indicates identical rank under R^2 and CRPS, while the highlighted Pareto-optimal point identifies EXAONE Tabular, which achieves the best mean rank for both metrics. Name tags are restricted to selected representative models.

distribution. The joint position of EXAONE Tabular at the leading corner therefore indicates that its strong point-estimation performance is accompanied by leading distributional performance.

The leading CRPS rank therefore shows that the point-estimation performance of EXAONE Tabular is not obtained at the expense of distributional quality. Its predictive distributions achieve the strongest aggregate agreement with the observed outcomes among the eligible models, while its predictive means simultaneously lead the R^2 and RMSE comparisons. Because CRPS combines multiple aspects of probabilistic prediction, this result should be interpreted as the best overall CRPS performance on the benchmark rather than as a standalone claim of perfect calibration. Together, these results demonstrate that EXAONE Tabular provides both accurate point estimates and high-quality distributional predictions across heterogeneous tabular regression tasks.

5 Limitations and Future Work

Class-Count Handling. The native classification head supports up to 10 classes. Datasets with larger label spaces are handled through an ECOC-based decomposition at inference time. This procedure requires multiple classification subproblem predictions and therefore increases inference cost as the number of classes grows. A class-count-independent prediction head is a potential direction for future work.

Large-Context Inference. Query chunking controls peak query-side memory because query predictions are conditionally independent given a fixed support context and compatible feature encoding. The inference pipeline can reuse support-side keys and values across query chunks when applicable, reducing redundant support-side computation. Otherwise, it uses uncached execution and recomputes the corresponding support-side values for each chunk. Although caching can improve inference efficiency, it introduces an additional memory cost, and its benefit depends on the support size, query size, and execution environment.

Support sets beyond the configured inference limit are currently subsampled. Potential future directions include context compression, representative-context selection, clustering-based support reduction, retrieval-based context construction, and adaptive support-set sampling. These methods require systematic evaluation of the trade-offs among inference latency, memory consumption, support compression, and predictive performance.

6 License and Permitted Use

The EXAONE Tabular release is distributed under separate licenses for the inference software and the model weights.

The `exaonetabular` inference runtime and associated source code released through the official GitHub repository are provided under the BSD-3-Clause-LG AI Research License. Subject to the terms of that license, the software may be used, modified, and redistributed, including for commercial purposes.

The released EXAONE Tabular model weights are licensed separately under the *EXAONE AI Model License Agreement 1.2 - NC*. The model license permits use of the weights for non-commercial research purposes, subject to the terms and restrictions specified in the license agreement. Commercial use of the released model weights requires separate authorization from the licensor.

The complete and authoritative license terms are distributed with the corresponding software and model weights through the official GitHub and Hugging Face repositories. This section provides only a high-level summary and does not replace, modify, or supersede those license terms.

7 Conclusion

We introduced EXAONE Tabular, a compact tabular foundation model family for classification and regression based on in-context learning. Its central architectural contribution is the Cross-Axis Summary Transformer (CAST), which interleaves within-row feature processing with support-conditioned across-row contextualization throughout the Transformer hierarchy. Distinct item-summary and feature-summary tokens mediate these interactions: item summaries join feature-axis sequences, while feature summaries join support item-axis sequences, allowing the model to retain cell-level representations while progressively refining feature interactions with support-set context.

The classification and regression models are trained separately on synthetic tasks generated entirely from an SCM prior and perform prediction without dataset-specific gradient updates. The resulting models contain approximately 20.8M and 21.1M parameters, respectively, while supporting native missing-value handling, task-specific prediction heads, test-time ensembling, and support-conditioned chunked inference.

Across four public benchmarks, EXAONE Tabular demonstrates consistently strong performance across classification, point regression, and probabilistic regression. On TabArena, the classification model ranks first among all evaluated methods using a single untuned default configuration, while regression reaches the performance regime of the 1.64B-parameter TabFM at roughly 1/11 the inference cost. On BCCO and TALENT, EXAONE Tabular ranks second in classification and first in regression. On ScoringBench, it achieves the best mean rank for R^2, RMSE, and CRPS, demonstrating leading performance for both point estimation and predictive-distribution quality. Together, these results establish EXAONE Tabular as a compact and highly competitive foundation model family across a broad range of real-world tabular prediction settings.

References

[1] Minh-Khoi Pham et al. “Retrieval-Aligned Tabular Foundation Models Enable Robust Clinical Risk Prediction in Electronic Health Records Under Real-World Constraints”. In: *arXiv preprint arXiv:2604.01841* (2026).

[2] Jillian M. Clements et al. “Sequential Deep Learning for Credit Risk Monitoring with Tabular Financial Data”. In: *arXiv preprint arXiv:2012.15330* (2020).

[3] Ida Hector and Rukmani Panjanathan. “Predictive Maintenance in Industry 4.0: A Survey of Planning Models and Machine Learning Techniques”. In: *PeerJ Computer Science* 10 (2024), e2016.

[4] Abdul Quadir Md et al. “A Review on Data-Driven Quality Prediction in the Production Process with Machine Learning for Industry 4.0”. In: *Processes* 10.10 (2022), p. 1966.

[5] Vahid Attari and Raymundo Arroyave. “Decoding Non-Linearity and Complexity: Deep Tabular Learning Approaches for Materials Science”. In: *Digital Discovery* (2025). arXiv:2411.18717.

[6] Vadim Borisov et al. “Deep Neural Networks and Tabular Data: A Survey”. In: *IEEE Transactions on Neural Networks and Learning Systems* 35.6 (2024), pp. 7499–7519.

[7] Tianqi Chen and Carlos Guestrin. “XGBoost: A Scalable Tree Boosting System”. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2016, pp. 785–794. DOI: 10.1145/2939672.2939785.

[8] Guolin Ke et al. “LightGBM: A Highly Efficient Gradient Boosting Decision Tree”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.

[9] Liudmila Prokhorenkova et al. “CatBoost: Unbiased Boosting with Categorical Features”. In: *Advances in Neural Information Processing Systems*. Vol. 31. 2018.

[10] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. “Why Do Tree-Based Models Still Outperform Deep Learning on Typical Tabular Data?” In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022, pp. 507–520.

[11] Samuel Müller et al. “Transformers Can Do Bayesian Inference”. In: *International Conference on Learning Representations*. 2022.

[12] Noah Hollmann et al. “TabPFN: A Transformer That Solves Small Tabular Classification Problems in a Second”. In: *International Conference on Learning Representations*. 2023.

[13] Noah Hollmann et al. “Accurate Predictions on Small Data with a Tabular Foundation Model”. In: *Nature* 637 (2025), pp. 319–326. DOI: 10.1038/s41586-024-08328-6.

[14] Jingang Qu et al. “TabICL: A Tabular Foundation Model for In-Context Learning on Large Data”. In: *arXiv preprint arXiv:2502.05564* (2025). Published at ICML 2025.

[15] Léo Grinsztajn et al. “TabPFN-3: Technical Report”. In: *arXiv preprint arXiv:2605.13986* (2026).

[16] Jingang Qu et al. “TabICLv2: A Better, Faster, Scalable, and Open Tabular Foundation Model”. In: *arXiv preprint arXiv:2602.11139* (2026).

[17] Nick Erickson et al. “TabArena: A Living Benchmark for Machine Learning on Tabular Data”. In: *arXiv preprint arXiv:2506.16791* (2025).

[18] Google Research. *Introducing TabFM: A Zero-Shot Foundation Model for Tabular Data*. Google Research Blog. June 2026. URL: https://research.google/blog/introducing-tabfm-a-zero-shot-foundation-model-for-tabular-data/.

[19] Xingxuan Zhang et al. “LimiX: Unleashing Structured-Data Modeling Capability for Generalist Intelligence”. In: *arXiv preprint arXiv:2509.03505* (2025).

[20] Si-Yang Liu et al. “TALENT: A Tabular Analytics and Learning Toolbox”. In: *arXiv preprint arXiv:2407.04057* (2024).

[21] Jonas Landsgesell, Pascal Knoll, and Tizian Wenzel. *ScoringBench: A Benchmark for Evaluating Tabular Foundation Models with Proper Scoring Rules*. 2026. arXiv: 2603.29928 [cs.AI].

[22] Léo Grinsztajn et al. “TabPFN-2.5: Advancing the State of the Art in Tabular Foundation Models”. In: *arXiv preprint arXiv:2511.08667* (2025).

[23] Junwei Ma et al. “TabDPT: Scaling Tabular Foundation Models”. In: *arXiv preprint arXiv:2410.18164* (2024).

[24] Ken M. Nakanishi. “Scalable-Softmax Is Superior for Attention”. In: *arXiv preprint arXiv:2501.19399* (2025).

[25] Thomas G. Dietterich and Ghulum Bakiri. “Solving Multiclass Learning Problems via Error-Correcting Output Codes”. In: *Journal of Artificial Intelligence Research* 2 (1995), pp. 263–286. DOI: 10.1613/jair.105.

[26] Tri Dao et al. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022.

[27] Markus N. Rabe and Charles Staats. “Self-attention Does Not Need $O(n^2)$ Memory”. In: *arXiv preprint arXiv:2112.05682* (2021).

[28] Jingyuan Liu et al. “Muon Is Scalable for LLM Training”. In: *arXiv preprint arXiv:2502.16982* (2025).

[29] Ilya Loshchilov and Frank Hutter. “Decoupled Weight Decay Regularization”. In: *International Conference on Learning Representations*. 2019.

[30] Kaiyue Wen et al. “Understanding Warmup-Stable-Decay Learning Rates: A River Valley Loss Landscape Perspective”. In: *arXiv preprint arXiv:2410.05192* (2024).

[31] Si-Yang Liu and Han-Jia Ye. “TabSwift: An Efficient Tabular Foundation Model with Row-Wise Attention”. In: *arXiv preprint arXiv:2606.07345* (2026).

[32] Nick Erickson et al. “AutoGluon-Tabular: Robust and Accurate AutoML for Structured Data”. In: *arXiv preprint arXiv:2003.06505* (2020).

[33] Synthefy. *Nori: Regression-Specialized Tabular Foundation Models*. Model release. 2026. URL: https://huggingface.co/Synthefy/Nori-30M.

[34] David Holzmüller, Léo Grinsztajn, and Ingo Steinwart. “Better by Default: Strong Pre-Tuned MLPs and Boosted Trees on Tabular Data”. In: *Advances in Neural Information Processing Systems*. Vol. 38. 2024.